CAB BOOKING SYSTEM

Aims: Microservices – Real-time – Event-driven – AI-enabled – Zero Trust Architecture

---

Tóm tắt (Abstract)

Tài liệu này trình bày việc phân tích, thiết kế và đánh giá một hệ thống ứng dụng đặt xe taxi hiện đại dựa trên kiến trúc Microservices, đáp ứng các yêu cầu về khả năng mở rộng, độ tin cậy, xử lý thời gian thực, bảo mật theo mô hình Zero Trust và tích hợp t rí tuệ nhân tạo. Hệ thống được thiết kế hướng cloud-native, sử dụng các công nghệ phổ biến trong công nghiệp nhằm đảm bảo tính khả thi triển khai thực tế.

Technology Stack:

Back-end: NodeJS / ExpressJS / NestJS

Front-end: React JS / Next JS + Tailwind CSS

Deployment: AWS Cloud / Terraform/ Kubernetes

Event-driven: Kafka, RabbitMQ

Communication: gRPC, restful api

Real-time: web socket , hook

Security: Zero trust Architecture

Monitor: Grafana Prometheus

Logging: ELK

Industrial University of Ho Chi Minh City

Mục lục (Table of Contents)

1. Giới thiệu
2. Phân tích bài toán & yêu cầu hệ thống
3. Tổng quan kiến trúc hệ thống
4. Kiến trúc Microservices
5. Kiến trúc Real-time & Event-driven
6. Kiến trúc AI & Intelligent Services
7. Kiến trúc bảo mật (Zero Trust Architecture)

8. Kiến trúc mở rộng & chịu lỗi (Scalability & Resilience)
9. Thiết kế chi tiết (Sequence Diagrams)
10. Thiết kế dữ liệu (ERD)
11. API Specification (OpenAPI)
12. Failure Scenarios & Recovery Strategies
13. Saga Pattern cho Payment
14. Stack công nghệ & Chi phí triển khai
15. Kết luận

THIẾT KẾ KIẾN TRÚC HỆ THỐNG ỨNG DỤNG ĐẶT XE TAXI

CAB BOOKING SYSTEM

Tóm tắt điều hành (Executive Summary)

Tài liệu này mô tả **thiết kế kiến trúc hoàn chỉnh** cho hệ thống ứng dụng đặt xe Taxi (CAB Booking System) xây dựng theo **kiến trúc Microservices**, sử dụng **Node.js** cho backend và **React.js** cho frontend. Hệ thống được thiết kế để đáp ứng các yêu cầu **real-time**, **mở rộng lớn**, **chịu lỗi cao**, **bảo mật**, và sẵn sàng triển khai trên **Docker Swarm / Cloud & Kubernetes**.

Tài liệu phù hợp để sử dụng cho:

- Hồ sơ **thiết kế hệ thống (System Design Document)**
- Khóa luận tốt nghiệp đại học chuyên ngành Hệ thống thông tin
- Tài liệu kỹ thuật cho team phát triển & vận hành

---

1. Tổng quan hệ thống

Hệ thống CAB Booking là nền tảng kết nối **khách hàng – tài xế – nhà vận hành** theo thời gian thực, hỗ trợ:

- Đặt xe
- Định vị GPS real-time
- Ghép tài xế thông minh

- Thanh toán
- Đánh giá & quản trị

Kiến trúc áp dụng **Microservices + Event-driven**, cho phép từng dịch vụ mở rộng độc lập và giảm phụ thuộc lẫn nhau.

---

2. Mục tiêu & Nguyên tắc thiết kế

2.1 Mục tiêu

- Scalability: mở rộng theo lưu lượng
- High Availability: hệ thống luôn sẵn sàng
- Fault Tolerance: không có single point of failure (SPOF)
- Real-time: cập nhật trạng thái & vị trí tức thời
- Cloud-native: Docker container / Pod, orchestration (Docker Swarm / Kubernetes), CI/CD (Github Action)

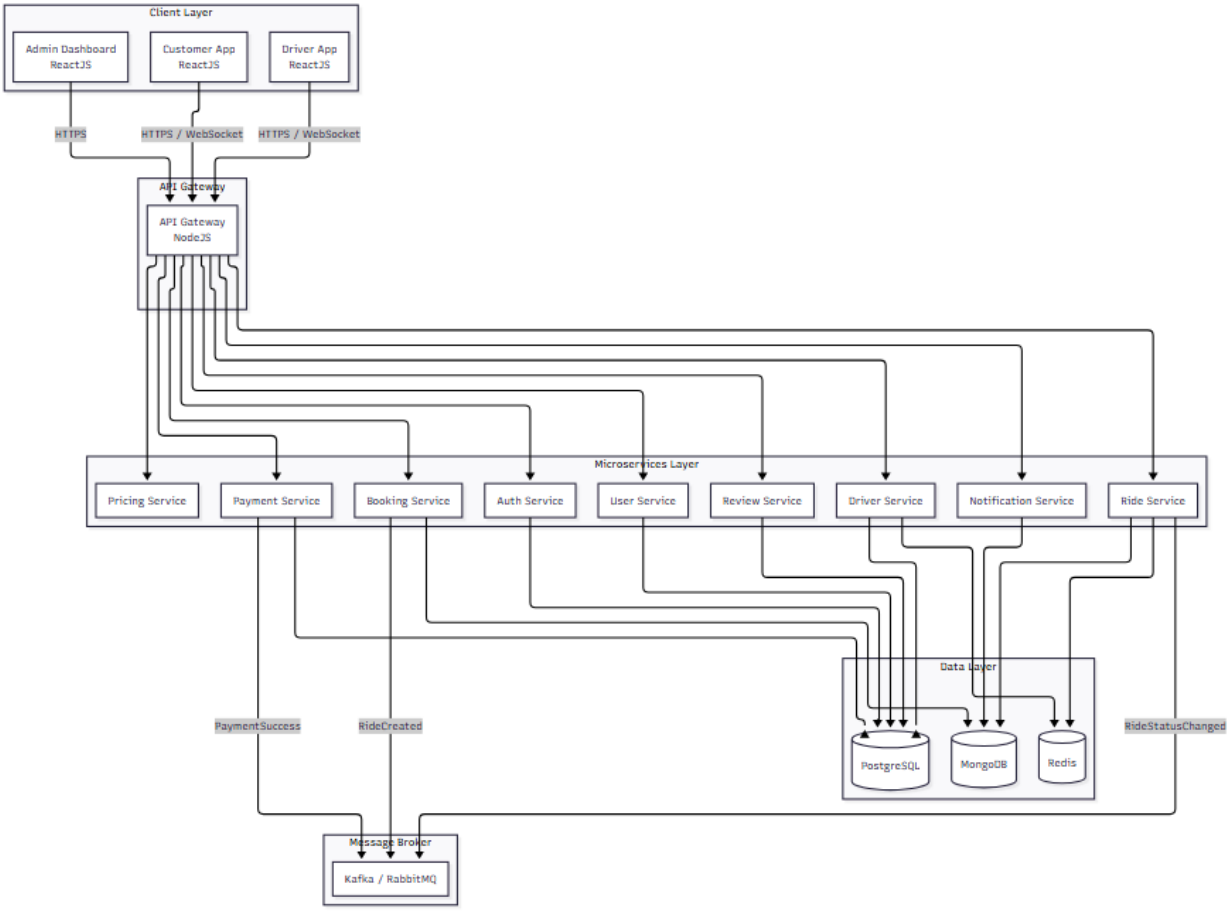
2.2 Nguyên tắc kiến trúc

- Database per service
- Stateless services
- Async-first (Event-driven)
- Zero trust security
- Observability by design

Industrial University of Ho Chi Minh City

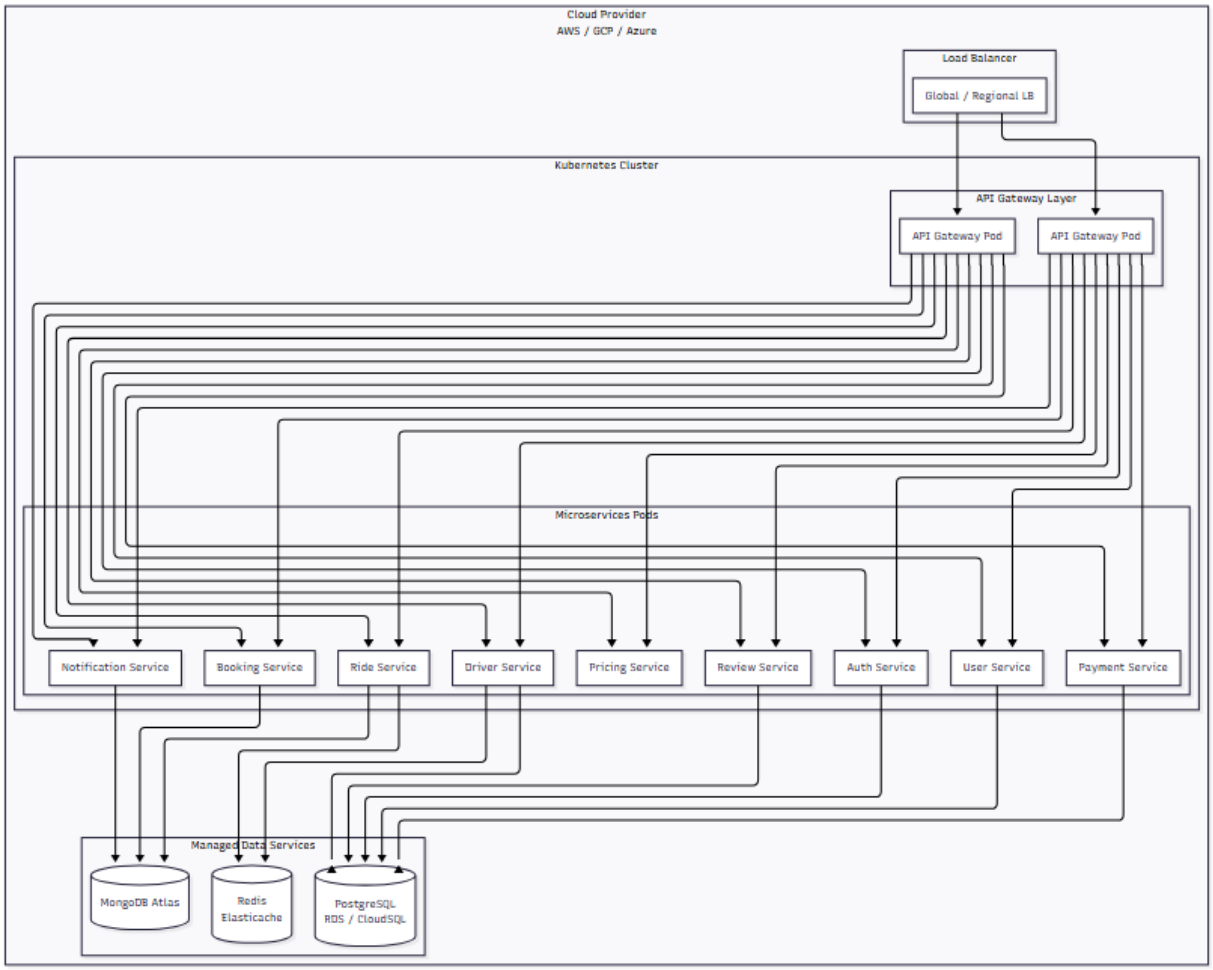
3. Kiến trúc tổng thể (Overall Architecture)

3.1 Kiến trúc tổng thể – Microservices

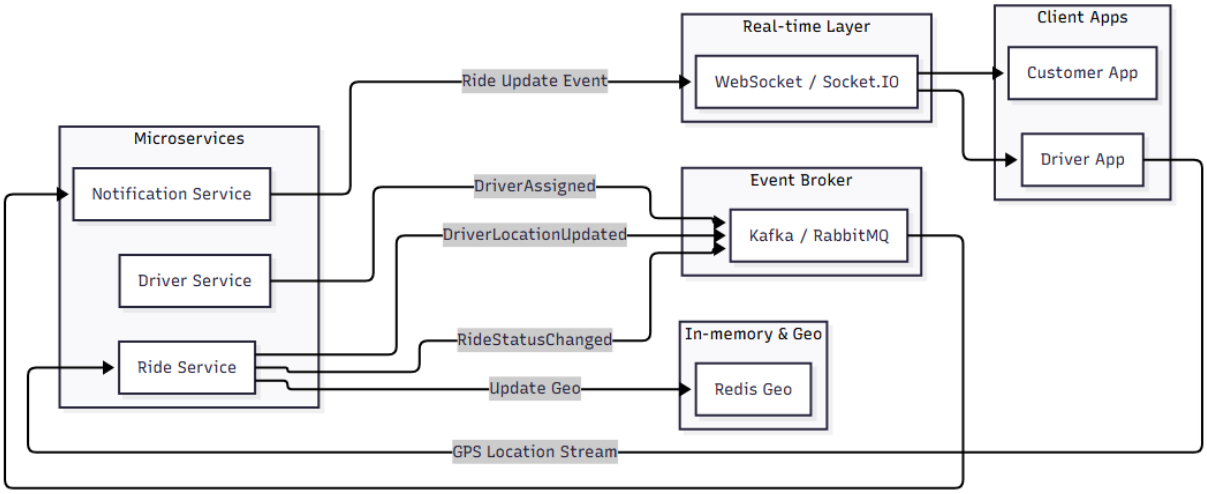


4. Kiến trúc triển khai (Deployment Architecture)

Hệ thống được triển khai trên **Docker Swarm on VirtualBox / Cloud + Terraform + Kubernetes**, hỗ trợ multi-region và auto-scaling.



5. Kiến trúc Real-time & Event-driven



---

6. Kiến trúc bảo mật & Zero Trust (Security / Zero Trust Architecture)

6.1 Nguyên lý Zero Trust áp dụng

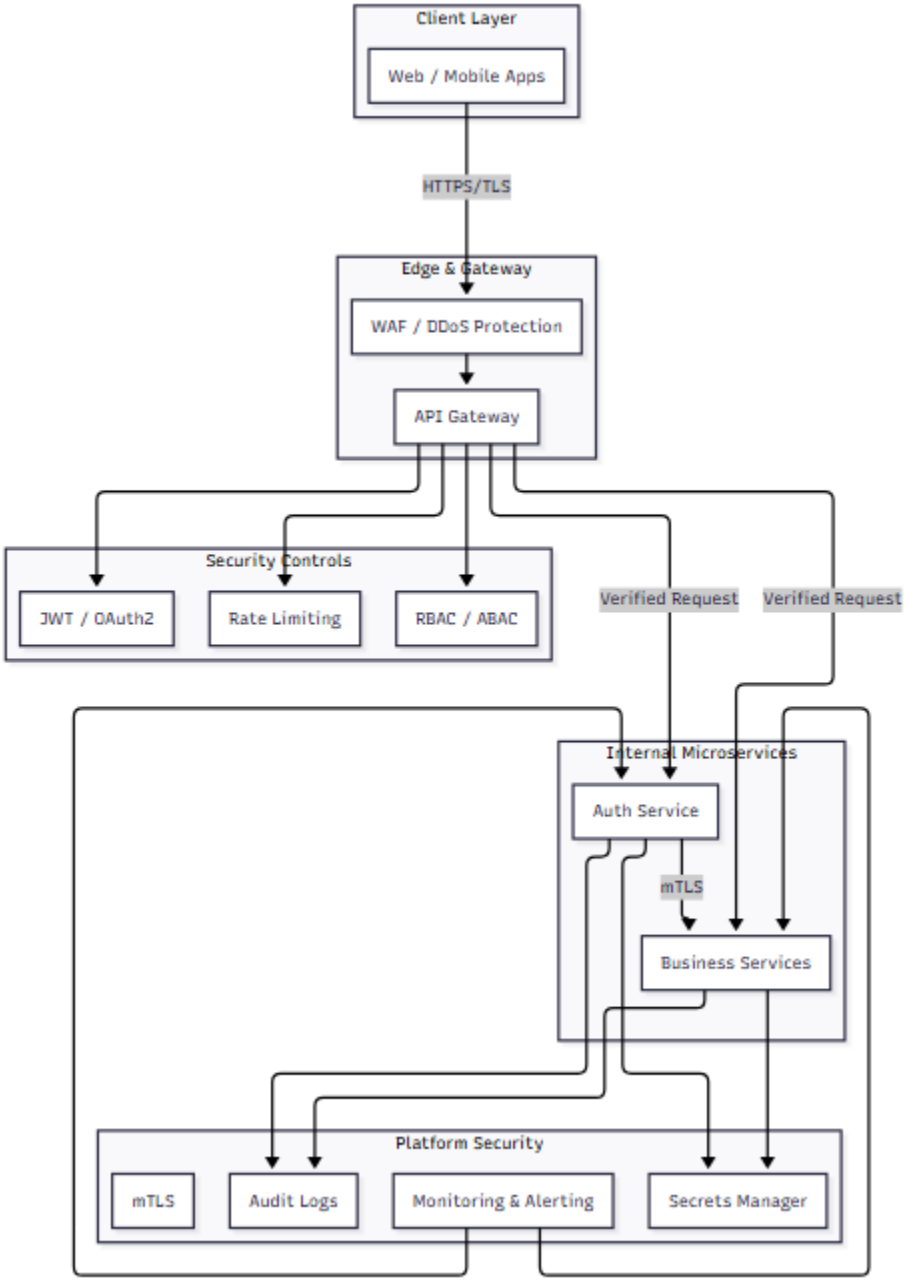
Kiến trúc bảo mật của hệ thống CAB Booking được thiết kế theo mô hình **Zero Trust**, với nguyên tắc cốt lõi:

- **Never trust, always verify**
- Mọi request đều phải được xác thực và ủy quyền
- Không giả định an toàn dù request đến từ mạng nội bộ

Zero Trust được áp dụng xuyên suốt từ **Client → Gateway → Microservices → Data**.

Industrial University of Ho Chi Minh City

6.2 Sơ đồ kiến trúc Zero Trust



6.3 Bảo mật tầng Client & Edge

- HTTPS/TLS 1.3 bắt buộc
- WAF chống:
  - SQL Injection
  - XSS
  - DDoS layer 7
- Rate limiting theo IP / user / device
- Device fingerprinting (tuỳ chọn)

6.4 API Gateway Security

API Gateway đóng vai trò **Policy Enforcement Point (PEP)**:

- Xác thực JWT / OAuth2
- Kiểm tra scope, role, permission
- Rate limit & quota
- Request validation (schema)
- Chặn request bất thường trước khi vào service

Industrial University of Ho Chi Minh City

6.5 Service-to-Service Security

- **mTLS** cho toàn bộ giao tiếp nội bộ
- Mỗi service có **service identity** riêng
- Áp dụng thông qua:
  - Service Mesh (Istio / Linkerd)

Lợi ích:

- Ngăn lateral movement
- Mutual authentication
- Encrypted traffic nội bộ

6.6 Authorization: RBAC & ABAC

- RBAC cho quyền cơ bản (Customer / Driver / Admin)
- ABAC cho ngữ cảnh động:
  - Thời gian
  - Vị trí
  - Trạng thái chuyến đi

Ví dụ: Driver chỉ được cập nhật GPS khi ride đang ACTIVE

6.7 Identity & Access Management (IAM)

- Central Auth Service
- JWT ngắn hạn + Refresh Token
- Token rotation
- Token revocation (Redis blacklist)
- MFA cho Admin

6.8 Secrets Management & Key Management

- Không hard-code secrets
- Sử dụng:
  - HashiCorp Vault / Cloud Secret Manager
- Key rotation định kỳ
- Mã hóa dữ liệu nhạy cảm (PII, payment)

6.9 Data Security & Privacy

- Encryption at-rest & in-transit
- Masking dữ liệu nhạy cảm
- Tách dữ liệu theo service (database per service)
- Tuân thủ:

Industrial University of Ho Chi Minh City

- GDPR / PDPA (ở mức kiến trúc)

6.10 Audit, Logging & Threat Detection

- Audit log cho:
  - Login
  - Payment
  - Permission change
- Centralized logging (ELK / OpenSearch)
- SIEM phát hiện bất thường
- Alert real-time cho sự cố bảo mật

6.11 Mapping Zero Trust → Failure Scenarios

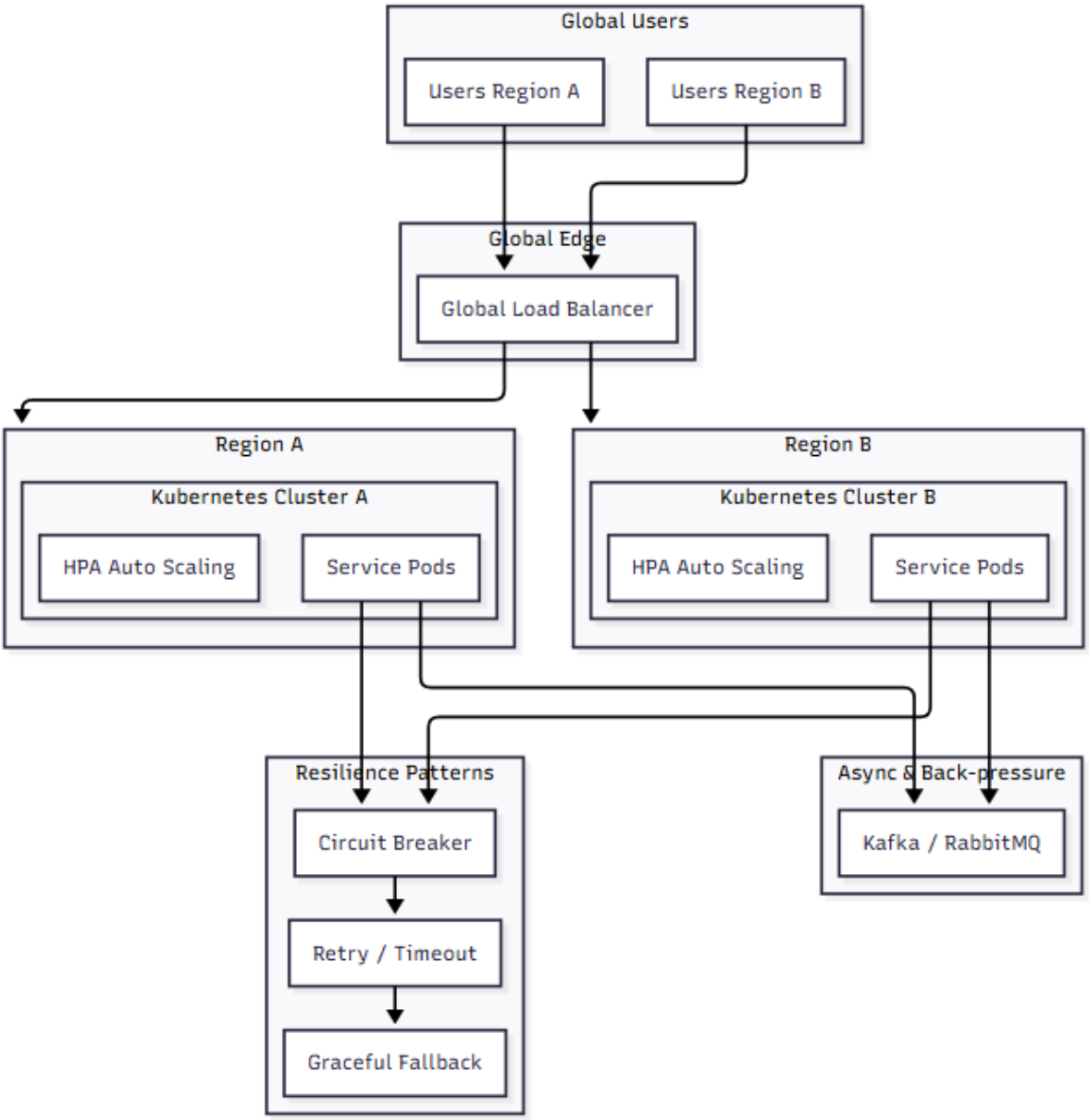
Nguy cơ	Cơ chế Zero Trust
Token bị lộ	Token rotation + revoke
Service compromise	mTLS + isolation
Lateral movement	Service identity
Insider threat	Audit & SIEM
DDoS	WAF + rate limit

6.12 Đánh giá học thuật

Mô hình Zero Trust phù hợp với hệ thống **microservices phân tán**, đặc biệt trong môi trường cloud, nơi ranh giới mạng truyền thống không còn hiệu quả. Việc áp dụng Zero Trust giúp giảm đáng kể rủi ro tấn công chuỗi và tăng khả năng kiểm soát truy cập.

Industrial University of Ho Chi Minh City

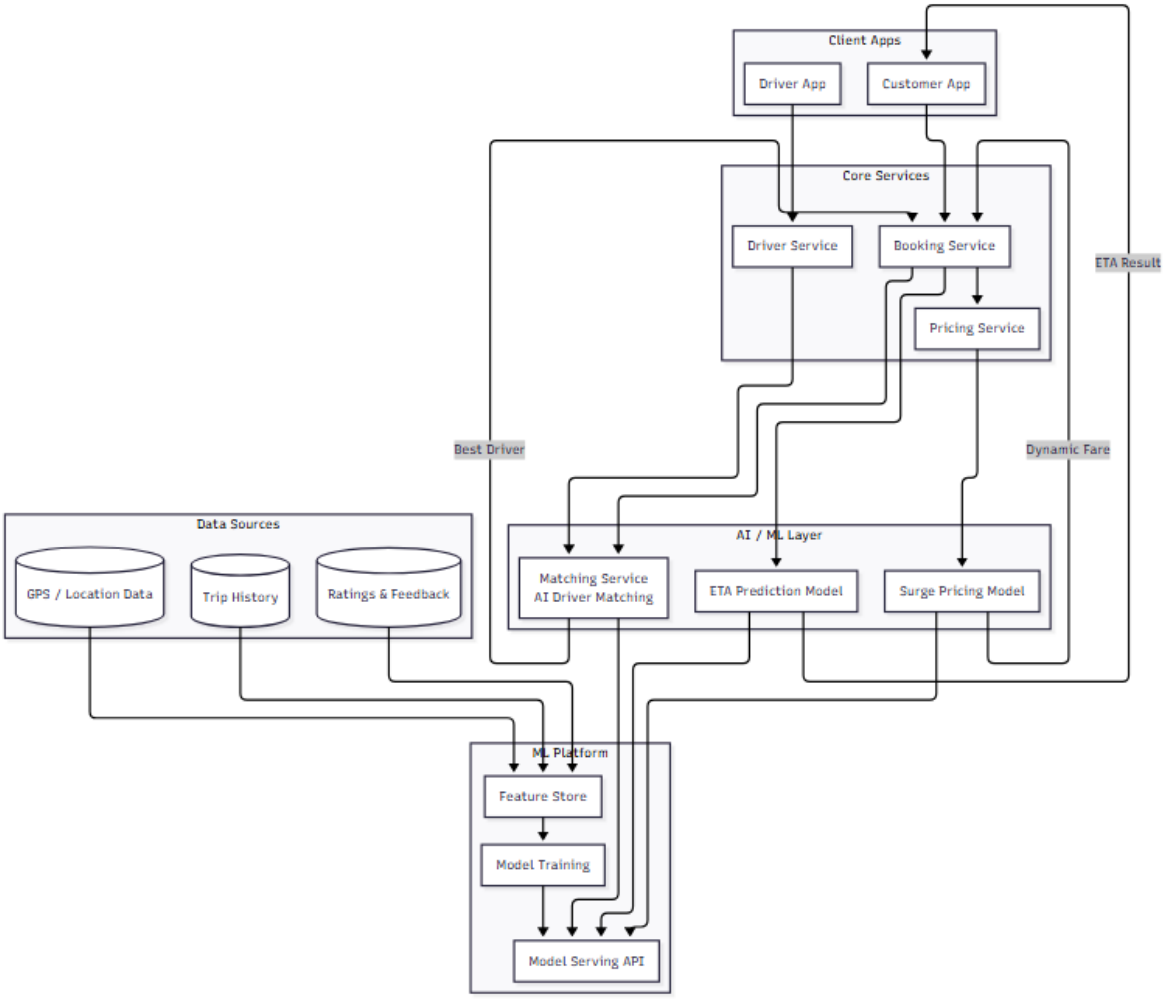
7. **Kiến trúc mở rộng & chịu lỗi (Scalability & Resilience)**



Áp dụng các pattern:

- Horizontal Pod Autoscaling (HPA)
- Circuit Breaker
- Retry / Timeout
- Graceful Degradation
- Eventual Consistency

8. Kiến trúc AI & Machine Learning



Ứng dụng AI:

- Ghép tài xế tối ưu
- Surge pricing
- Dự đoán ETA
- Gợi ý & phân tích hành vi

Các use-case AI chính

1. **AI Driver Matching**
    - Chọn tài xế tối ưu dựa trên:
        - Khoảng cách (GPS)

        - Lịch sử chuyến đi
        - Rating & hành vi

2. **Surge Pricing**
    - Giá động theo:
        - Cung – cầu
        - Thời gian, khu vực
        - Lịch sử dữ liệu
3. **ETA Prediction**
    - Dự đoán thời gian xe đến chính xác hơn

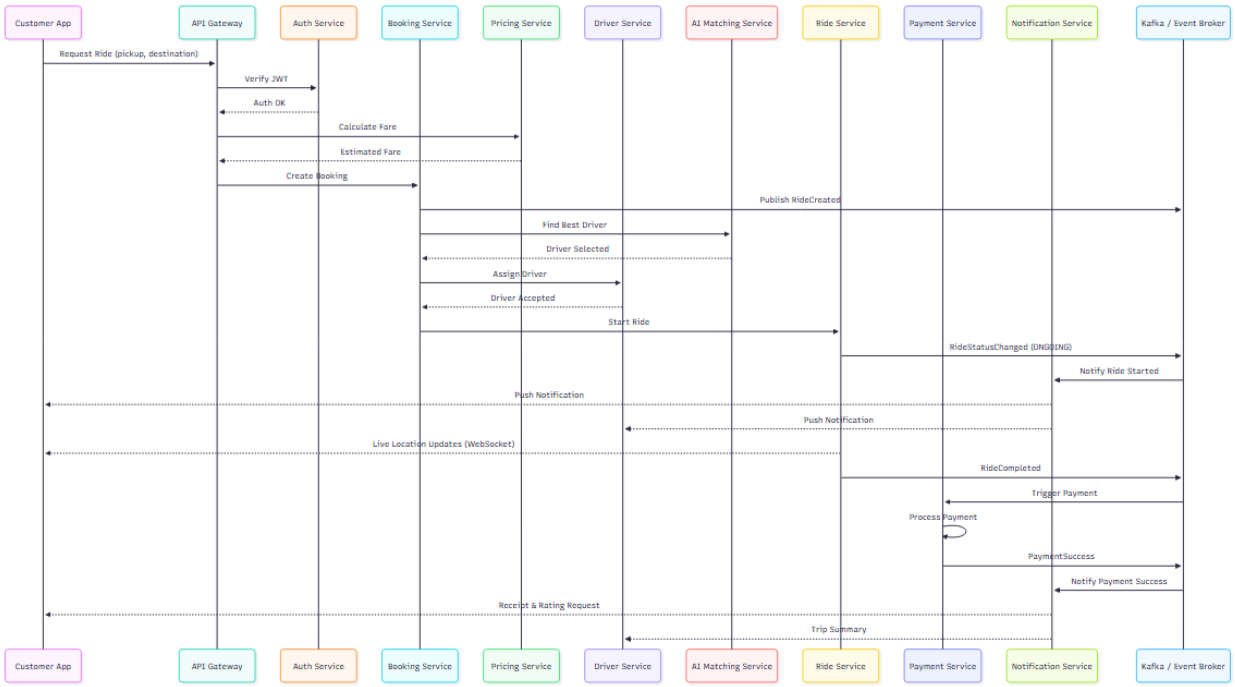
Kiến trúc AI trong sơ đồ

- **AI/ML Layer**:
    - Matching Service
    - Surge Pricing Model
    - ETA Prediction Model
- **ML Platform**:
    - Feature Store
    - Model Training
    - Model Serving API
- **Data Sources**:
    - Trip History
    - GPS / Location
    - Rating & Feedback

Industrial University of Ho Chi Minh City

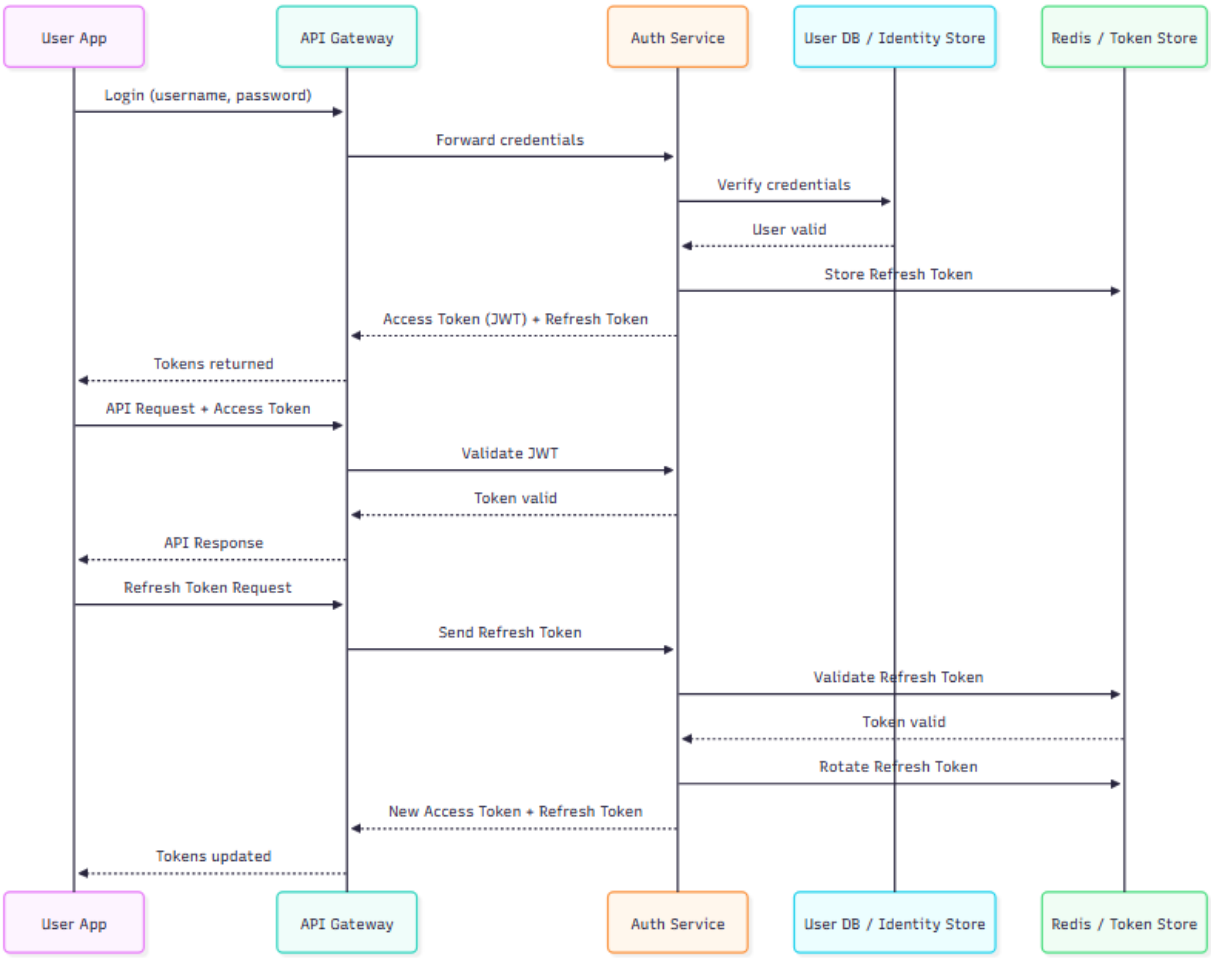
9. Sequence Diagram

9.1 Quy trình đặt xe end-to-end



University of Ho Chi Minh City

9.2 Login & Refresh Token



Sequence Diagram này thể hiện rõ 3 luồng quan trọng

1 Login Flow

- User gửi **username/password**
- API Gateway → Auth Service
- Auth Service:
  - Verify DB / Identity Store
  - Sinh **Access Token (JWT)** + **Refresh Token**
  - Lưu Refresh Token vào **Redis**

2 Gọi API được bảo vệ

- User gửi **Access Token**

- Gateway xác thực JWT với Auth Service
- Cho phép request đi tiếp

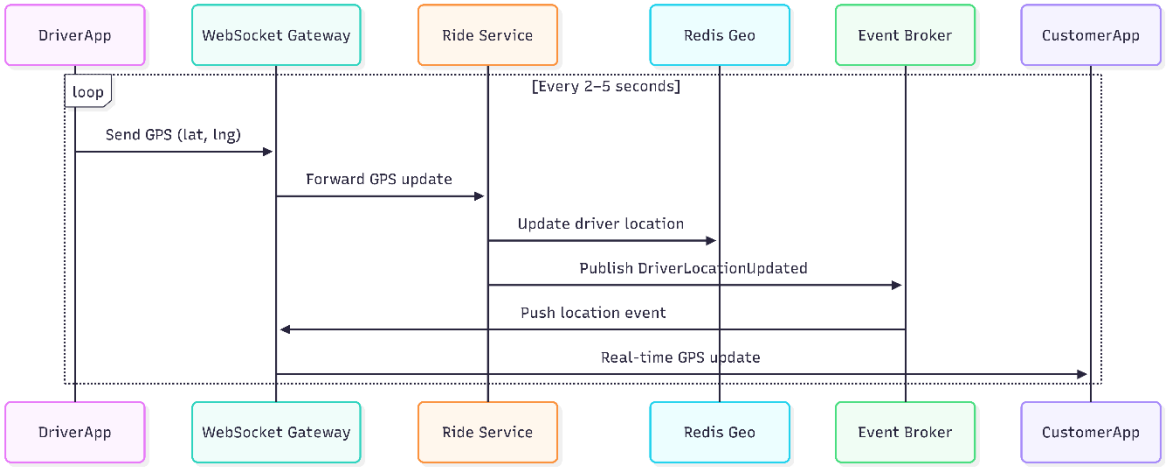
3️⃣ **Refresh Token Flow**

- Access Token hết hạn
- User gửi **Refresh Token**
- Auth Service:
  - Kiểm tra Redis
  - **Rotate Refresh Token** (best practice)
- Trả về cặp token mới

👉 Đây là **chuẩn bảo mật enterprise** (OWASP khuyến nghị), rất phù hợp để:

- Giải thích kiến trúc Security
- Bảo vệ hệ thống CAB real-time

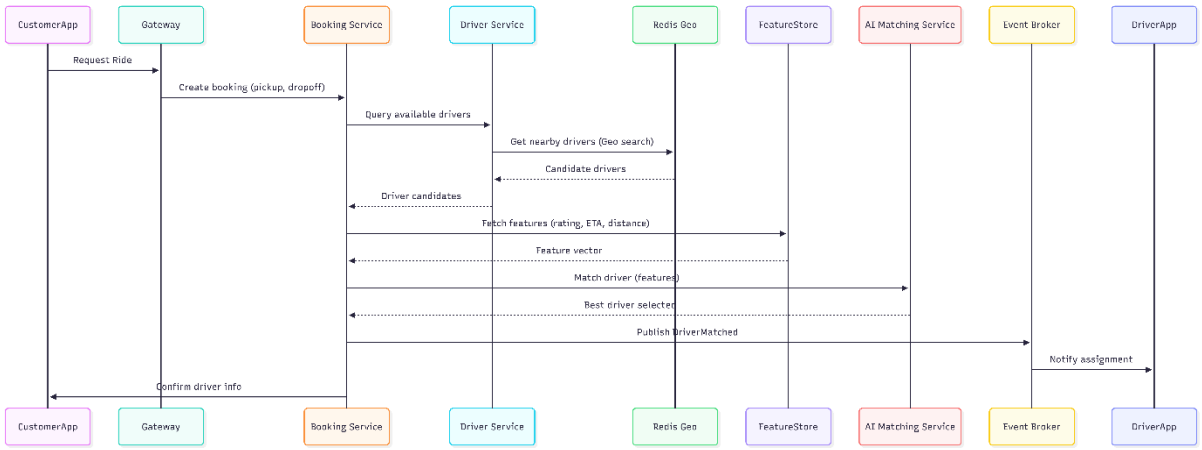
9.3 Real-time GPS Update (Driver → Passenger)



Giải thích:

- Driver gửi GPS định kỳ qua WebSocket
- Ride Service cập nhật Redis (Geo index) để query nhanh
- Event được publish để các service khác consume (ETA, AI, Monitoring)
- Passenger nhận vị trí gần real-time (<1s latency)

9.4 AI Driver Matching (Chi tiết)

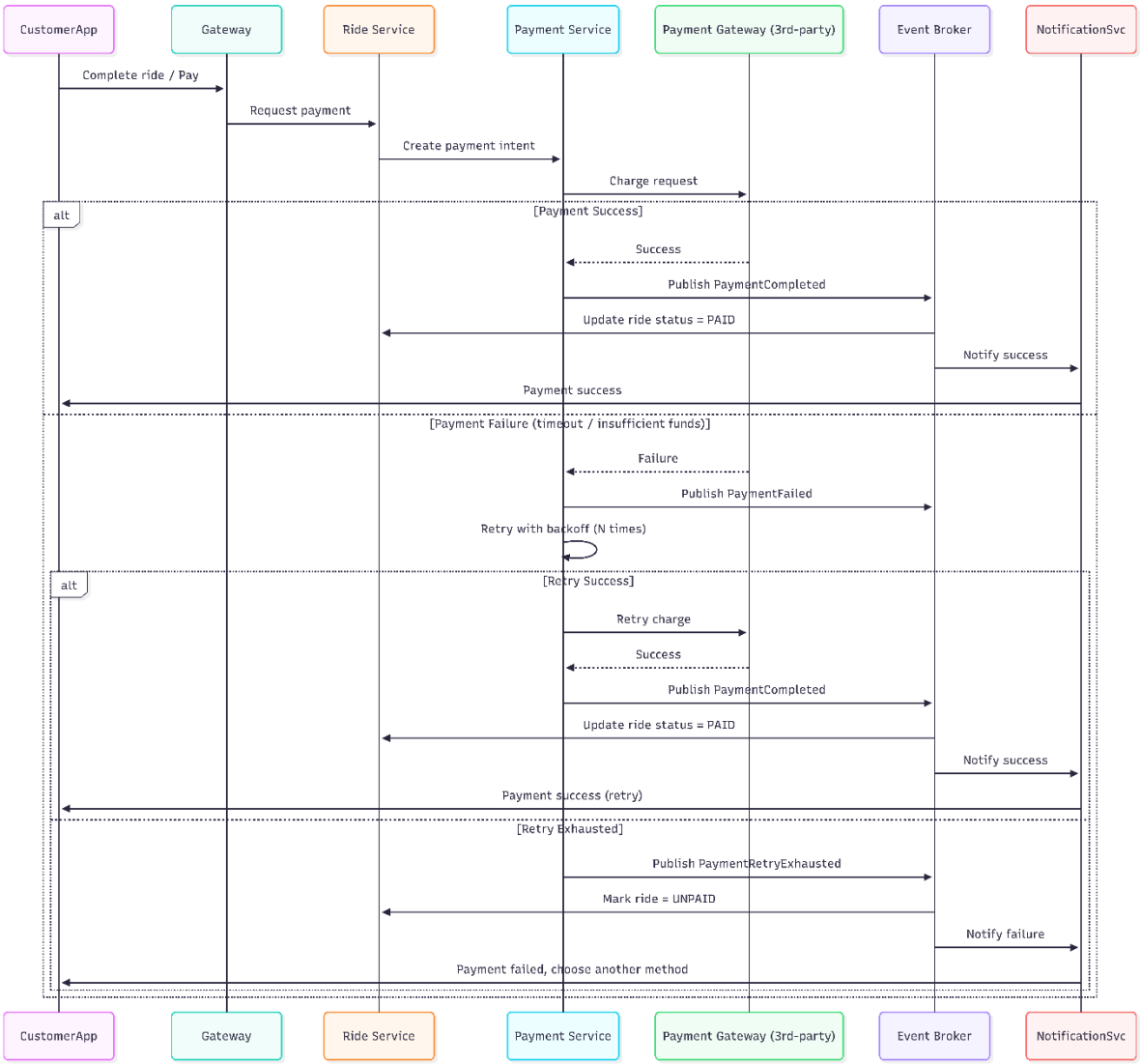


Giải thích chi tiết:

- Redis Geo dùng để lọc **tài xế gần nhất** (hard constraint)
- Feature Store cung cấp dữ liệu cho AI (rating, lịch sử, ETA)
- AI Matching Service xử lý **soft constraints & scoring**
- Kết quả được publish qua Kafka để đảm bảo **decoupling**
- Cho phép fallback sang rule-based matching khi AI lỗi

Industrial University of Ho Chi Minh City

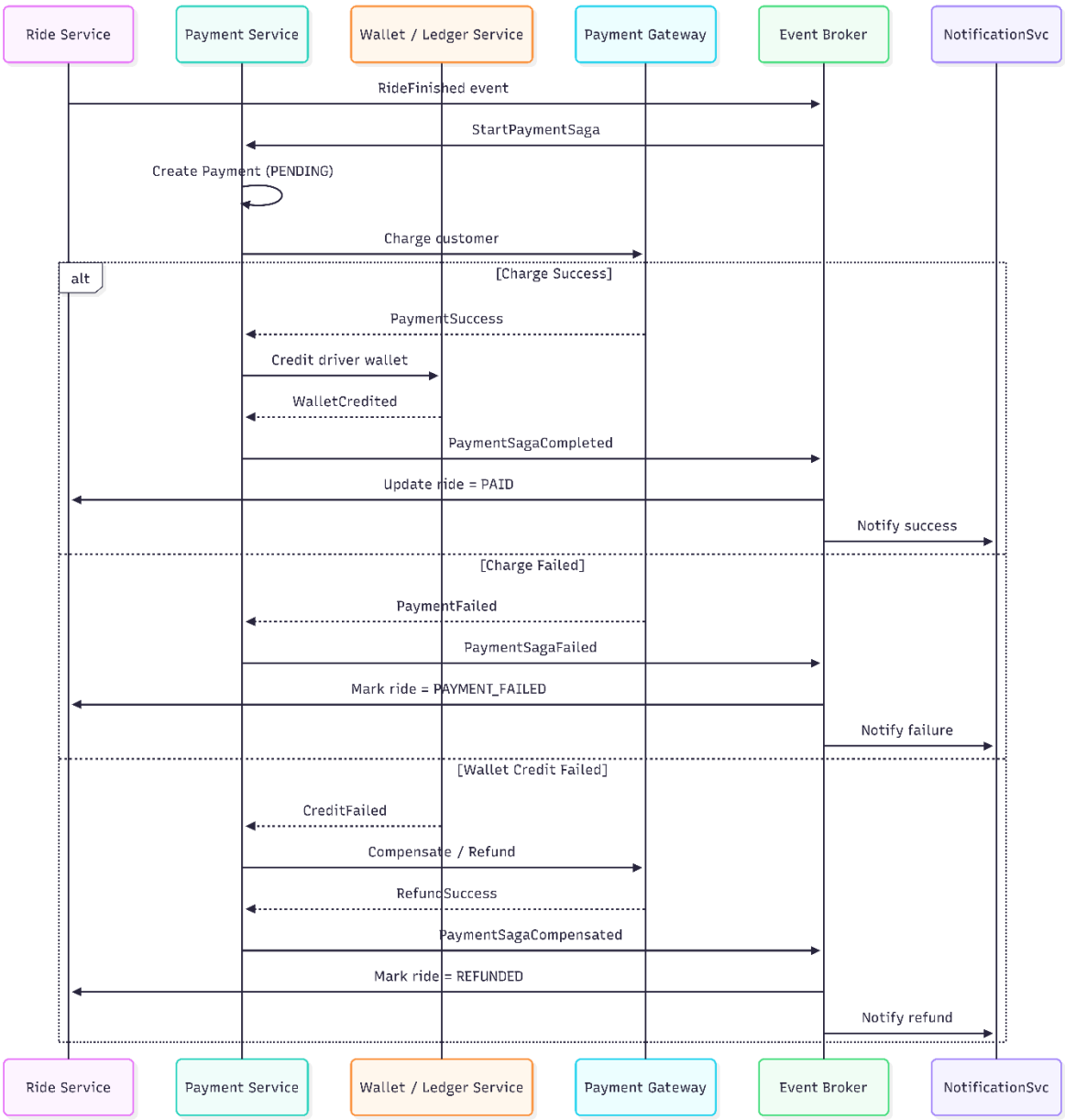
9.5 Payment Failure & Retry (Chi tiết)



Giải thích:

- Áp dụng **Retry + Exponential Backoff** cho payment
- Payment Service là **source of truth** cho trạng thái thanh toán
- Event-driven đảm bảo **eventual consistency**
- Không block luồng người dùng khi cổng thanh toán chậm
- Cho phép mở rộng sang nhiều PSP (Stripe, VNPay, MoMo, …)

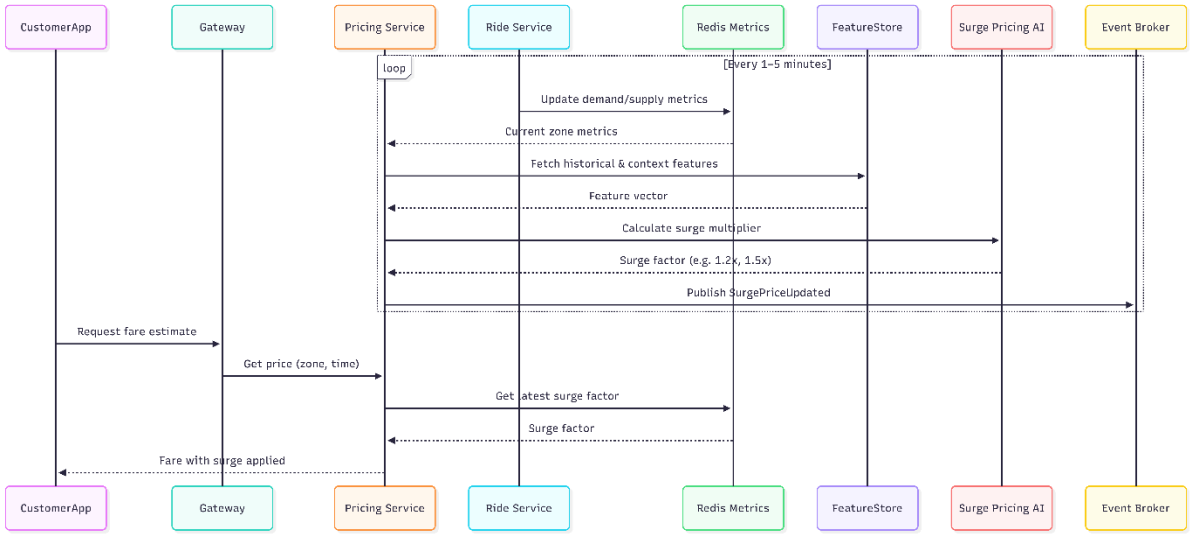
9.5.1 Saga Pattern cho Payment (Hoàn chỉnh)



Đặc điểm Saga Payment:

- Saga **choreography-based** (event-driven)
- Không sử dụng distributed transaction (2PC)
- Mỗi bước có **compensation action** rõ ràng
- Đảm bảo **no double charge – no lost money**
- Phù hợp với payment real-world (PSP không reliable)

9.6 Surge Pricing Real-time (Chi tiết)

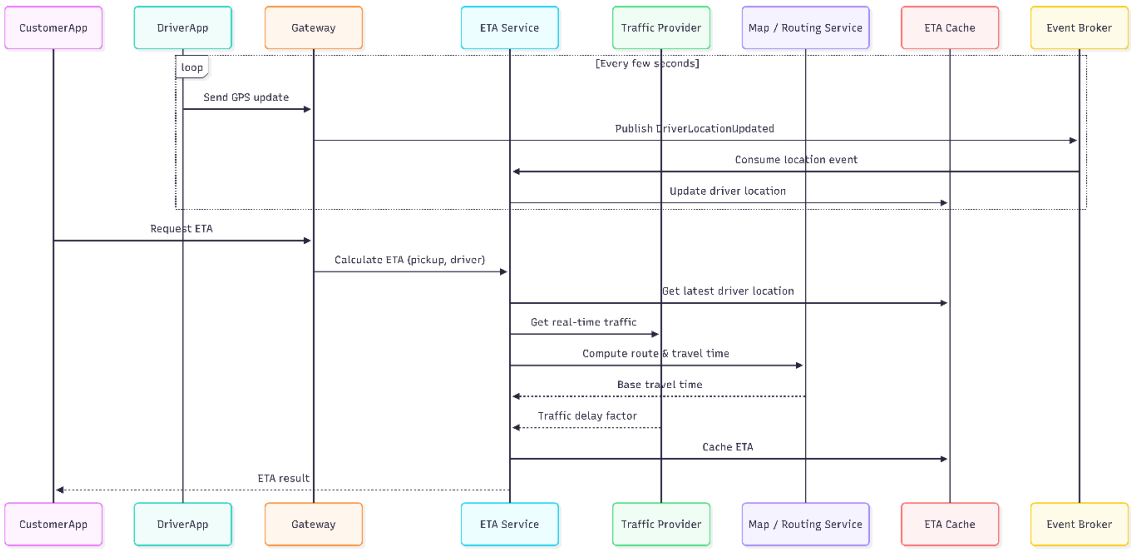


Giải thích chi tiết:

- Surge Pricing chạy **near real-time**, tách khỏi flow booking
- Redis lưu metrics cung/cầu theo zone để truy xuất nhanh
- AI model xử lý yếu tố thời gian, sự kiện, lịch sử
- Kafka broadcast surge change cho cache, dashboard, analytics
- Đảm bảo **giá nhất quán** giữa estimate và booking

Industrial University of Ho Chi Minh City

9.7 ETA Calculation Real-time (Chi tiết)



Điểm kiến trúc nổi bật

- **ETA là service độc lập (**xử lý **tính toán độc lập)** → không block booking/matching (tránh làm chậm booking)
- **GPS cập nhật event-driven** (Kafka) → real-time đúng nghĩa
- **Redis làm hot-store** cho vị trí & ETA (Redis dùng cache ETA & vị trí) → giảm latency , latency thấp
- **Tách routing & traffic** → dễ thay Google Map / Here / OSRM
- **Mở đường cho AI ETA** (bias correction theo lịch sử). Có thể tích hợp AI để điều chỉnh ETA theo lịch sử.

9A. Thiết kế UI/UX cho hệ thống CAB Booking

Phần này bổ sung thiết kế **UI/UX ở mức kiến trúc & wireframe**, phục vụ mục tiêu:

- Hoàn chỉnh luận văn (end-to-end system)
- Chứng minh tính khả thi triển khai frontend
- Làm cầu nối giữa kiến trúc backend và trải nghiệm người dùng

Thiết kế UI tuân theo các nguyên tắc:

- Mobile-first
- Real-time feedback

- Role-based UI (Customer / Driver / Admin)
- Tối giản – dễ mở rộng

9A.1 Tổng quan các ứng dụng giao diện

Ứng dụng	Người dùng	Mục tiêu
Customer App	Hành khách	Đặt xe, theo dõi, thanh toán
Driver App	Tài xế	Nhận chuyến, dẫn đường, thu nhập
Admin Dashboard	Vận hành	Giám sát, cấu hình, phân tích

9A.2 UI/UX – Customer App (Hành khách)

Phần này trình bày **thiết kế UI chi tiết ở mức màn hình & component**, đủ để:

- Làm phụ lục thiết kế giao diện cho luận văn
- Chuyển trực tiếp sang Figma / React implementation

9A.2.1 Nguyên tắc thiết kế

- Mobile-first (iOS / Android)
- One-hand usage
- Real-time feedback (GPS, ETA, price)
- Progressive disclosure (ẩn bớt chi tiết nâng cao)

9A.2.2 Danh sách màn hình Customer App

ID	Màn hình	Mục tiêu
C1	Splash / Onboarding	Giới thiệu & quyền truy cập
C2	Login / Register	Xác thực người dùng
C3	Home – Map & Pickup	Đặt điểm đón
C4	Destination	Nhập điểm đến

Industrial University of Ho Chi Minh City

ID	Màn hình	Mục tiêu
C5	Ride Options	Chọn loại xe & giá
C6	Searching Driver	Matching real-time
C7	Ride Tracking	Theo dõi chuyến đi
C8	Payment	Thanh toán
C9	Rating & Feedback	Đánh giá
C10	Ride History	Lịch sử
C11	Profile & Wallet	Thông tin cá nhân

9A.2.3 UI chi tiết từng màn hình

C1 – Splash / Onboarding

- Logo trung tâm
- 3 slide giới thiệu (Book – Track – Pay)
- CTA: Get Started

Components:

- Carousel
- Primary Button

C2 – Login / Register

Layout:

- Phone / Email input
- OTP verification
- Social login (optional)

Components:

- InputField
- OTPInput

Industrial University of Ho Chi Minh City

- SubmitButton

API:

- Auth Service (Login / Refresh Token)

C3 – Home (Map & Pickup)

Layout:

- Full-screen Map
- Pickup pin (center)
- Bottom Sheet (collapsed)

Bottom Sheet content:

- Pickup address (auto-detect)
- Shortcut: Home / Work
- CTA: Set Destination

Components:

- MapView
- LocationSearch
- BottomSheet

WebSocket:

- Driver nearby

C4 – Destination Selection

Layout:

- Search bar
- Suggested places
- Recent destinations

Components:

- SearchInput

Industrial University of Ho Chi Minh City

- PlaceList

C5 – Ride Options & Pricing

Layout:

- Horizontal ride cards
- ETA + price
- Surge indicator

Ride Card:

- Vehicle icon
- Capacity
- ETA
- Price

Components:

- RideOptionCard
- PriceBreakdownModal

API:

- Pricing Service
- ETA Service

C6 – Searching Driver (Matching)

Layout:

- Map + animated ripple
- Status text
- Cancel button

States:

- Searching
- Driver Found

Industrial University of Ho Chi Minh City

WebSocket:

- Matching updates

C7 – Ride Tracking (In-progress)

Layout:

- Map (driver + route)
- Driver info card
- ETA live update

Driver Card:

- Avatar
- Rating
- Call / Chat

Components:

- DriverInfoCard
- LiveRouteMap

Industrial University of Ho Chi Minh City

C8 – Payment

Layout:

- Fare summary
- Payment method selector
- Pay button

Methods:

- Cash
- Card
- Wallet

API:

- Payment Service

C9 – Rating & Feedback

Layout:

- Star rating
- Comment box
- Tip (optional)

Components:

- RatingStars
- TextArea

C10 – Ride History

Layout:

- List view
- Filter by date/status

Item:

- Date
- Route summary
- Amount

C11 – Profile & Wallet

Tabs:

- Profile
- Wallet
- Settings

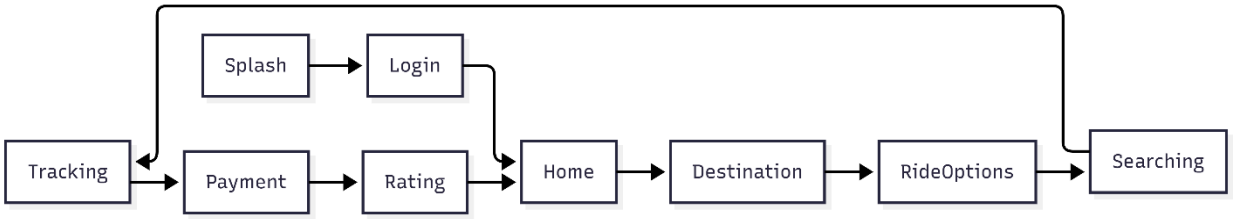
Features:

- Saved locations
- Payment methods

Industrial University of Ho Chi Minh City

- Logout

9A.2.4 Luồng UI tổng thể (Customer Journey)



9A.2.5 Mapping UI Components → React Structure

```
/pages
  /auth
  /home
  /ride
  /payment
  /profile
/components
  MapView
  BottomSheet
  RideOptionCard
  DriverInfoCard
/services
  authApi.ts
  rideApi.ts
  paymentApi.ts
```

Industrial University of Ho Chi Minh City

9A.2.6 Đánh giá

Thiết kế UI Customer App được chi tiết hóa đến mức **component & API mapping**, đảm bảo:

- Phù hợp triển khai React / React Native
- Hỗ trợ real-time, AI matching, surge pricing
- Đủ chiều sâu cho phụ lục thiết kế giao diện trong luận văn

9A.3 UI/UX – Driver App (Tài xế)

Các màn hình chính

1. Login / KYC
2. Online / Offline toggle
3. Nhận chuyến (Incoming request)
4. Dẫn đường tới điểm đón
5. Theo dõi chuyến đi
6. Kết thúc & thu nhập
7. Lịch sử chuyến đi

Wireframe luồng Driver

Thành phần UI

- Map full-screen
- Accept / Reject CTA lớn
- Voice navigation
- Real-time income widget

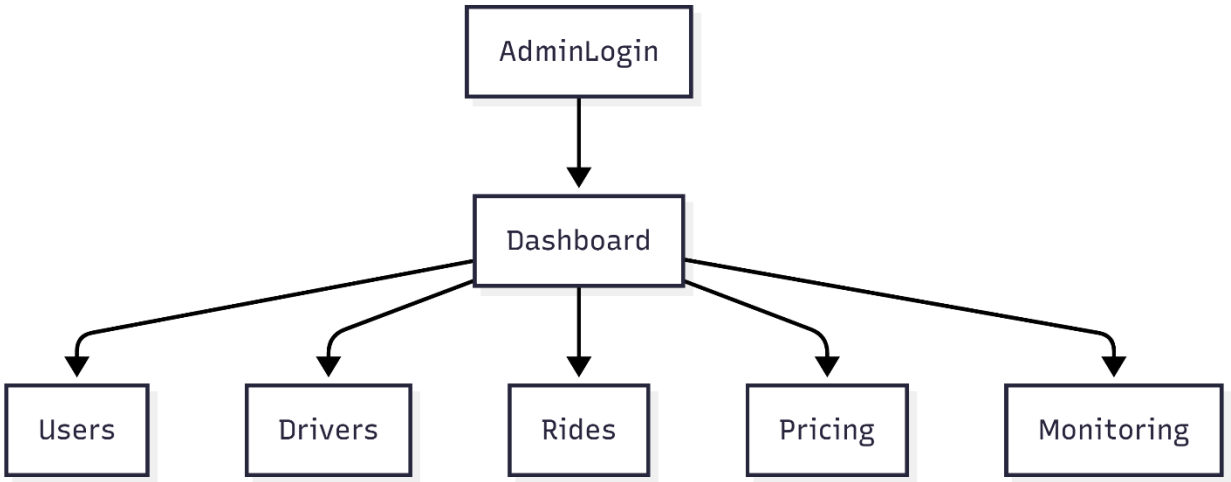
University of Ho Chi Minh City

9A.4 UI/UX – Admin Dashboard

Các module chính

- Dashboard tổng quan (KPI)
- Quản lý người dùng & tài xế
- Quản lý chuyến đi
- Giám sát real-time map
- Pricing & surge control
- Logs & audit

Sơ đồ module Admin



9A.5 Mapping UI ↔ Backend Services

UI Action	API / Service
Login	Auth Service
Đặt xe	Booking Service
Matching	AI Matching Service
Theo dõi GPS	Ride + WebSocket
ETA	ETA Service
Thanh toán	Payment Service
Surge price	Pricing Service

9A.6 Công nghệ Frontend đề xuất

- ReactJS / NextJS + TypeScript
- React Query / Redux Toolkit
- Socket.IO client
- Mapbox / Google Maps SDK

Industrial University of Ho Chi Minh City

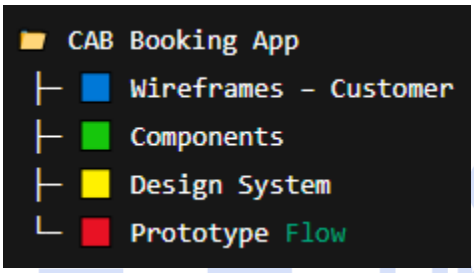
- Tailwind / MUI
- PWA / React Native (mở rộng)

9A.7 Đánh giá học thuật

Thiết kế UI/UX được trình bày ở mức **conceptual & architectural**, phù hợp với phạm vi luận văn. Việc mô hình hóa luồng người dùng và mapping với backend microservices giúp chứng minh tính nhất quán end-to-end của hệ thống, đồng thời đảm bảo khả năng triển khai thực tế.

9A.8 Design UI with Figma wireframe

Tiếp theo, sử dụng Figma kết hợp AI để thiết kế UI. Sau cùng, dùng AI tool để biến đổi thiết kế UI trong Figma thành React Component.



Tham khảo lý thuyết về UI/UX: https://www.figma.com/resource-library/what-is-wireframing/

---

10. Phương pháp nghiên cứu & tiếp cận luận văn

10.1 Phương pháp nghiên cứu

Luận văn áp dụng phương pháp **Design Science Research (DSR)**, trong đó trọng tâm là việc **thiết kế – xây dựng – đánh giá** một hiện vật (artifact) là hệ thống kiến trúc CAB Booking.

Các bước chính:

1. Xác định vấn đề thực tiễn (scalability, real-time, AI)
2. Đề xuất kiến trúc giải quyết vấn đề
3. Triển khai mô hình kiến trúc (conceptual design)
4. Đánh giá thông qua phân tích kiến trúc & kịch bản vận hành

10.2 Phạm vi nghiên cứu

- Tập trung vào **kiến trúc phần mềm & hệ phân tán**
- Không đi sâu vào UI/UX chi tiết

- AI ở mức kiến trúc & pipeline, không huấn luyện model cụ thể

---

11. Đánh giá kiến trúc (Architecture Evaluation)

11.1 Đánh giá theo thuộc tính chất lượng (Quality Attributes)

Thuộc tính	Giải pháp kiến trúc
Scalability	Microservices, HPA, Kafka
Availability	Multi-region, stateless services
Performance	Redis cache, async processing
Security	JWT, mTLS, Zero Trust
Maintainability	Service isolation, API contract

11.2 Kịch bản đánh giá (ATAM – rút gọn)

- Tăng 10x số lượng booking → hệ thống scale ngang
- Một service AI bị lỗi → fallback rule-based
- Payment gateway timeout → retry & eventual consistency

Industrial University of Ho Chi Minh City

---

12. Failure Scenarios & Fault Handling (Hoàn chỉnh)

Phần này mô tả **toàn bộ các kịch bản lỗi quan trọng** trong hệ thống CAB Booking, cách phát hiện, cô lập và phục hồi nhằm đảm bảo **High Availability** và **Graceful Degradation**.

12.1 Nhóm lỗi Authentication & Security

Failure Scenario	Nguyên nhân	Cách xử lý
Auth Service down	Pod crash / overload	API Gateway trả 503, dùng circuit breaker, cho phép refresh token cache ngắn hạn
JWT hết hạn	Token expired	Tự động refresh token
Token bị lộ	Client compromise	Revoke token qua Redis blacklist

Failure Scenario	Nguyên nhân	Cách xử lý
Brute-force login	Attack	Rate limit + CAPTCHA

12.2 Nhóm lỗi Booking & Matching

Failure Scenario	Nguyên nhân	Cách xử lý
Không tìm thấy tài xế	Supply thấp	Retry mở rộng bán kính, fallback rule-based
AI Matching lỗi	Model / service down	Fallback sang matching theo khoảng cách
Booking duplicate	Network retry	Idempotency key
Booking service crash	Pod failure	Booking được replay từ Kafka

12.3 Nhóm lỗi Real-time & GPS

Failure Scenario	Nguyên nhân	Cách xử lý
Mất kết nối WebSocket	Mobile network	Auto-reconnect, fallback polling
GPS trễ / mất	Thiết bị	Dùng last-known-location
Kafka lag	High throughput	Scale consumer group

12.4 Nhóm lỗi ETA & Pricing

Failure Scenario	Nguyên nhân	Cách xử lý
Traffic API down	Third-party	Dùng historical average
ETA Service overload	Peak traffic	Cache Redis + rate limit
Surge AI lỗi	Model serving down	Fallback fixed rule
Inconsistent price	Race condition	Price snapshot theo booking

Industrial University of Ho Chi Minh City

12.5 Nhóm lỗi Payment

Failure Scenario	Nguyên nhân	Cách xử lý
Payment timeout	PSP chậm	Retry + exponential backoff
PSP down	Third-party outage	Switch PSP khác
Double charge	Retry race	Idempotency key
Payment pending	Network split	Eventual consistency

12.6 Nhóm lỗi Data & Storage

Failure Scenario	Nguyên nhân	Cách xử lý
PostgreSQL fail	Node crash	Read replica + failover
Redis eviction	Memory pressure	TTL + recompute
MongoDB lag	Replication delay	Read preference

Industrial University of Ho Chi Minh City

12.7 Nhóm lỗi Infrastructure

Failure Scenario	Nguyên nhân	Cách xử lý
Pod crash	Bug / OOM	Auto restart
Node down	Cloud issue	Reschedule pod
Region outage	Disaster	Multi-region failover
Network partition	ISP	Eventual consistency

12.8 Failure Handling Patterns Tổng hợp

- Circuit Breaker
- Retry + Backoff
- Bulkhead Isolation

- Idempotency
- Saga pattern
- Eventual consistency
- Graceful degradation

---

13. Phân tích so sánh & Trade-off

13.1 So sánh Monolithic vs Microservices

Tiêu chí	Monolithic	Microservices
Scale	Khó	Dễ
Độ phức tạp	Thấp ban đầu	Cao hơn
Phù hợp taxi	❌	✅

13.2 Trade-off chính

- Tăng độ phức tạp vận hành để đổi lấy khả năng mở rộng
- Eventual consistency thay cho strong consistency
- Chi phí infra cao hơn nhưng đáp ứng real-time

---

14. Stack công nghệ & Chi phí triển khai (Technology Stack & Cost Estimation)

14.1 Tổng quan lựa chọn stack

Stack công nghệ được lựa chọn theo tiêu chí:

- Cloud-native, phổ biến, dễ tuyển nhân sự
- Phù hợp hệ thống real-time, microservices
- Có hệ sinh thái mạnh & cộng đồng lớn

Industrial University of Ho Chi Minh City

14.2 Stack công nghệ chi tiết

14.2.1 Frontend

Thành phần	Công nghệ
Web App	ReactJS / NextJS, TypeScript
State management	Redux Toolkit / React Query
Real-time	Socket.IO / WebSocket
Build & Deploy	Vite / CI-CD

14.2.2 Backend / Microservices

Thành phần	Công nghệ
Runtime	Node.js (NestJS / Express)
API	REST + OpenAPI Swagger
Auth	OAuth2, JWT
Real-time	Socket.IO
Validation	Zod / Joi
Microservices internal communication	Synchronous: request-response, like REST API (HTTP/HTTPS), gRPC Asynchronous: event-driven, message queues (RabbitMQ, Kafka)

14.2.3 Event & Messaging

Thành phần	Công nghệ
Message Broker	Apache Kafka (hoặc RabbitMQ)
Streaming	Kafka Streams
Schema	Schema Registry

Industrial University of Ho Chi Minh City

14.2.4 Data Layer

Thành phần	Công nghệ
Transactional DB	PostgreSQL
NoSQL	MongoDB
Cache / Geo	Redis
Search (tuỳ chọn)	OpenSearch / Elastic

14.2.5 AI / ML

Thành phần	Công nghệ
Training	Python, PyTorch / TensorFlow
Serving	FastAPI / TorchServe
Feature Store	Feast
Pipeline	Airflow

Industrial University of Ho Chi Minh City

14.2.6 Infrastructure & DevOps

Thành phần	Công nghệ
Cloud	AWS / GCP / Azure
Container	Docker
Orchestration	Kubernetes
Service Mesh	Istio / Linkerd
IaC	Terraform
CI/CD	GitHub Actions / GitLab CI

14.2.7 Observability & Security

Thành phần	Công nghệ
Monitoring	Prometheus + Grafana
Logging	ELK / OpenSearch
Tracing	Jaeger
Secrets	Vault / Cloud Secret Manager
WAF	Cloud WAF

14.3 Ước lượng chi phí triển khai (mang tính tham khảo)

Giả định: **~100.000 MAU, 5.000 concurrent users**, triển khai 1 region chính

14.3.1 Chi phí hạ tầng hàng tháng (USD)

Hạng mục	Dịch vụ	Chi phí ước tính
Kubernetes cluster	EKS / GKE	300 – 500
Compute (Pods)	EC2 / GCE	800 – 1,200
Load Balancer	ALB / GLB	100 – 150
PostgreSQL (managed)	RDS / Cloud SQL	300 – 500
MongoDB	Atlas	200 – 400
Redis	ElastiCache	150 – 300
Kafka	MSK / Confluent	300 – 600
Storage & Backup	Object Storage	100 – 200
Observability	Logs + Metrics	100 – 200

Tổng ước tính: ~2,500 – 4,000 USD / tháng

Industrial University of Ho Chi Minh City

14.4 Chi phí mở rộng theo quy mô

Quy mô	MAU	Chi phí ước tính
Startup	10k	500 – 800 USD
Scale-up	100k	2,500 – 4,000 USD
Large scale	1M+	15,000+ USD

14.5 Trade-off chi phí

- Managed services tăng chi phí nhưng giảm rủi ro vận hành
- Kafka tốn kém hơn RabbitMQ nhưng phù hợp real-time scale lớn
- Multi-region tăng HA nhưng chi phí x2–x3
- AI inference chi phí cao → cần cache & batch

14.6 Đánh giá học thuật

Phân tích stack và chi phí cho thấy kiến trúc đề xuất **khả thi trong thực tế**, có thể triển khai từ quy mô startup đến enterprise. Việc ước lượng chi phí giúp đánh giá tính kinh tế (economic feasibility) của mô hình nghiên cứu.

15. Hướng phát triển tương lai (Future Work)

Mặc dù hệ thống đã được thiết kế tương đối hoàn chỉnh về mặt kiến trúc, bảo mật và khả năng mở rộng, vẫn còn nhiều hướng phát triển tiềm năng trong tương lai nhằm nâng cao hiệu năng, độ thông minh và khả năng thích ứng của hệ thống.

15.1 Nâng cao năng lực AI & Machine Learning

- Áp dụng **Reinforcement Learning** cho bài toán matching tài xế – khách hàng nhằm tối ưu hóa toàn cục (global optimization) thay vì tối ưu cục bộ.
- Sử dụng **Graph Neural Networks (GNN)** để mô hình hóa mạng lưới giao thông và hành vi tài xế.
- Triển khai **online learning** để mô hình tự thích nghi theo thời gian thực với thay đổi nhu cầu và điều kiện giao thông (Multi-modal transport by bike, delivery)
- Cá nhân hóa giá cước và ETA dựa trên hành vi người dùng (Graph-based ETA prediction)

15.2 Mở rộng kiến trúc dữ liệu & phân tích nâng cao

- Xây dựng **Data Lake / Lakehouse** phục vụ phân tích lớn và huấn luyện AI.

- Áp dụng **Real-time analytics** (Apache Flink / Spark Streaming) cho giám sát hệ thống và phát hiện bất thường.
- Phát triển **Feature Store dùng chung** cho nhiều mô hình AI.

15.3 Đa vùng & toàn cầu hóa hệ thống

- Triển khai **multi-region active-active** để giảm độ trễ và tăng khả năng chịu lỗi.
- Hỗ trợ **multi-currency, multi-language, multi-regulation**.
- Tối ưu định tuyến người dùng theo vị trí địa lý (Geo-routing).

15.4 Tăng cường bảo mật & quyền riêng tư

- Áp dụng **Confidential Computing** cho xử lý dữ liệu nhạy cảm.
- Nghiên cứu **Privacy-Preserving Machine Learning** (Federated Learning, Differential Privacy).
- Tự động hóa phản ứng sự cố bảo mật (SOAR).

15.5 Tối ưu chi phí & năng lượng

- Áp dụng **FinOps** để giám sát và tối ưu chi phí cloud.
- Sử dụng **spot/preemptible instances** cho workload AI.
- Nghiên cứu **Green IT** nhằm giảm tiêu thụ năng lượng.

15.6 Định hướng nghiên cứu học thuật

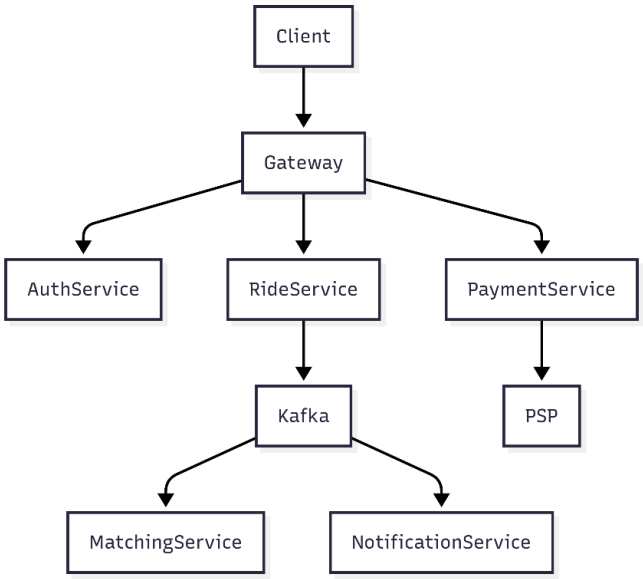
- So sánh hiệu quả kiến trúc Microservices với **Serverless / Event-native architectures**.
- Đánh giá tác động của các mô hình AI khác nhau đến độ công bằng (fairness) trong phân phối chuyến đi.
- Nghiên cứu khả năng áp dụng **Digital Twin** cho mô phỏng hệ thống giao thông.

Industrial University of Ho Chi Minh City

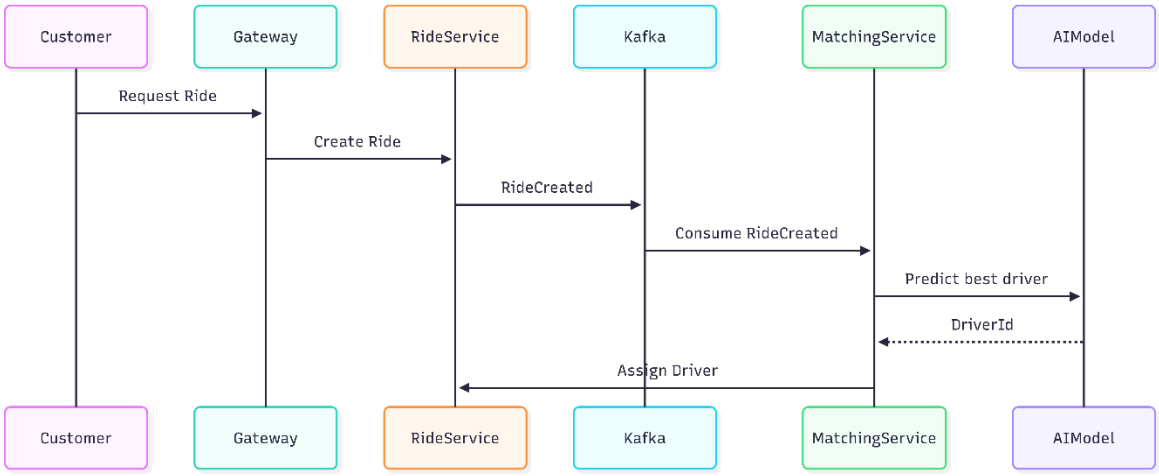
16. Phụ lục (Appendix)

Phụ lục A – Mermaid Diagrams

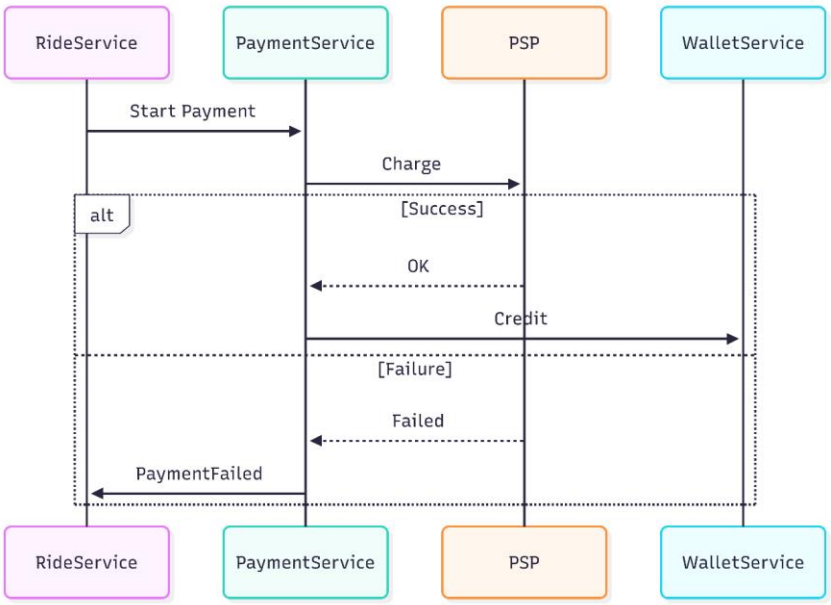
A.1 Tổng thể kiến trúc hệ thống



A.2 Sequence Diagram – AI Matching



A.3 Sequence Diagram – Payment Saga



Phụ lục B – OpenAPI Specification (FULL)

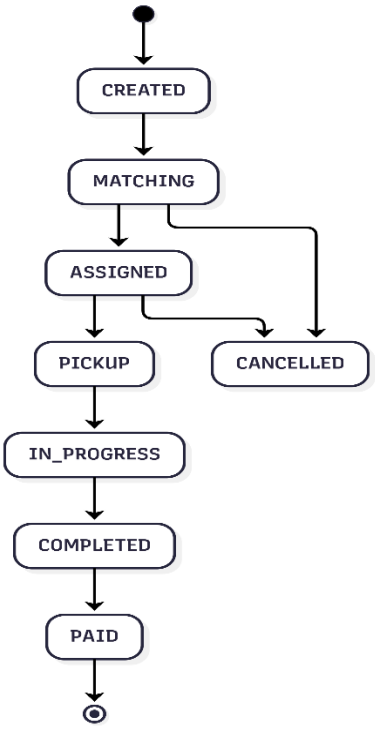
Xây dựng mô tả Restful API với OpenAPI 3.0 (OAS yaml file) – đầy đủ tất cả services, có thể import trực tiếp Swagger / Postman

default

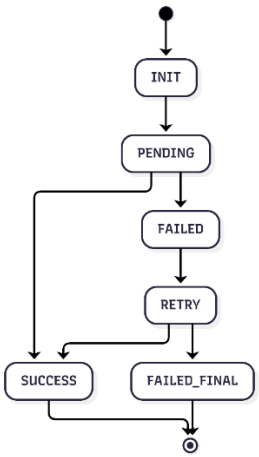
POST	/auth/login	Login
POST	/auth/refresh	Refresh token
GET	/users/profile	Get user profile
POST	/drivers/location	Update driver GPS
POST	/bookings	Create booking
POST	/rides/{id}/start	Start ride
POST	/rides/{id}/complete	Complete ride
POST	/payments/charge	Charge payment
POST	/reviews	Submit review

Phụ lục C – State Machine

C.1 Ride State Machine



C.2 Payment State Machine



Industrial University of Ho Chi Minh City

Phụ lục D – Kafka Topics & Event Schema

Topic	Producer	Consumer
ride.created	Booking	Matching, ETA
ride.assigned	Matching	Notification
driver.location.updated	Ride	ETA, Monitoring
payment.completed	Payment	Ride, Wallet
payment.failed	Payment	Notification

Sample Event (ride.created)

```
{
"eventId": "uuid",
  "type": "RideCreated",
  "rideId": "r123",
  "pickup": {"lat":10.7,"lng":106.6},
  "timestamp": "2025-01-01T10:00:00Z"
}
```

Industrial University of Ho Chi Minh City

Phụ lục E – Threat Model (STRIDE)

STRIDE	Threat	Mitigation
Spoofing	Token giả	JWT + mTLS
Tampering	Sửa payload	TLS + HMAC
Repudiation	Chối giao dịch	Audit log
Information Disclosure	Lộ PII	Encryption
DoS	Flood API	Rate limit, WAF
Elevation of Privilege	Leo quyền	RBAC/ABAC

inprotected.com